

1. תיאור כללי

התוכנית מממשת מנתח תחבירי (Parser) והיא נבנתה באמצעות Flex (עבור הניתוח הלקסיקלי) ו-Bison (עבור הניתוח התחבירי).

המנתח קורא קוד מקור מהקלט הסטנדרטי, מוודא את תקינותו בהתאם לדקדוק השפה ובונה עץ גזירה. אם הקלט תקין, התוכנית מדפיסה את מבנה העץ לפלט הסטנדרטי. במידה ומתגלה שגיאה, המנתח מדווח עליה ומסיים את הריצה עם קוד השגיאה המתאים.

2. מבני נתונים

מבנה הנתונים המרכזי המייצג את עץ הגזירה הוא המבנה ParserNode (המוגדר בקובץ part2_helpers.h).

מבנה הצומת (ParserNode)

העץ ממומש בשיטת Left-Child, Right-Sibling המאפשרת תמיכה יעילה בעצים בהם לצומת יכול להיות מספר משתנה של בנים:

```
typedef struct node {
    char *type; // סוג המשתנה התחבירי או שם האסימון) למשל "PROGRAM", "id"
    char *value; // ערך העזר (הלקסמה) עבור אסימונים רלוונטיים (למשל "55"), או NULL
    struct node *sibling; // מצביע לאח הבא (הצומת הבא באותה רמה)
    struct node *child; // מצביע לבן הראשון (הצאצא השמאלי ביותר)
} ParserNode;
```

- **עלים (Leaves):** נוצרים על ידי המנתח הלקסיקלי (part2.lex). הם מכילים את סוג האסימון (Token Type), ועבור אסימונים כגון מזהים (id), מספרים או מחרוזות, הם מכילים גם את הלקסמה (value).
- **צמתים פנימיים (Internal Nodes):** נוצרים על ידי המנתח התחבירי (part2.y). צמתים אלו מייצגים משתנים תחביריים (כגון EXP, STMT) ומאגדים תחתיהם את צמתי הבנים.

3. פרטי מימוש

ניתוח לקסיקלי (Flex)

המנתח הלקסיקלי סורק את הקלט וממיר רצפי תווים לאסימונים (Tokens).

- **טיפול במחרוזות:** עם פונקציית עזר unquote_string אשר מסירה את המרכאות הכפולות של המחרוזת בקלט, ושומרת בתוך הצומת (TK_STR) רק את תוכן המחרוזת עצמו, כנדרש בפורמט הפלט.
- **יצירת צמתים:** עבור כל אסימון מזהה, המנתח הלקסיקלי יוצר צומת ParserNode חדש ומעביר אותו למנתח התחבירי דרך המשתנה yylval.

ניתוח תחבירי (Bison)

המנתח התחבירי מגדיר את חוקי הגזירה ובונה את העץ בגישת Bottom-Up.

- **קישור צמתים (mkNodeN):** פונקציית עזר אשר מקבלת מספר משתנה של בנים, מקשרת אותם זה לזה באמצעות מצביעי ה-sibling, ולבסוף מחברת את ראש הרשימה למצביע ה-child של צומת האב החדש שנוצר.
- **גזירות ריקות (Epsilon):** עבור חוקי גזירה המייצגים ריק (כגון רשימת פקודות ריקה), יצרנו באופן מפורש צומת בשם <EPSILON> וחיברנו אותו לעץ.

4. טיפול בדו-משמעות וקדימויות

הגדרנו קדימויות (Precedence) ואסוציאטיביות ב-Bison, בהתאם למקובל בשפות ++C/C.

סדר הקדימויות (מהנמוך לגבוה)

1. **לוגיקה:** NOT > AND > OR (אסוציאטיביות לשמאל, למעט NOT לימין).
2. **יחס:** RELOP (כגון <, ==).
3. **חשבון:** ADDOP > MULOP (<, +, -, *).
4. **המרה (Casting):** המרה מפורשת EXP (TYPE).

פתרון קונפליקט המרה (Explicit Casting)

- קיים קונפליקט בין פעולת המרה לבין פעולות חשבון (למשל בביטוי $((int)a + b)$).
- **ההנחה:** פעולת ה-Cast חזקה יותר מחיבור או כפל.
 - **המימוש:** הוגדר אסימון דמה TK_CAST בעל הקדימות הגבוהה ביותר. כלל הגזירה של המרה סומן באמצעות `prec TK_CAST%`, כך שהביטוי הנ"ל יפורש כ- $((int)a) + b$.

פתרון בעיית ה-Dangling Else

- על מנת להבטיח ש-else ישוייך תמיד ל-if הקרוב ביותר אליו:
- השתמשנו בהגדרת `nonassoc% IF_NO_ELSE` עבור `IF_NO_ELSE` ועבור `TK_ELSE`.
 - לאסימון TK_ELSE ניתנה עדיפות גבוהה יותר, מה שכופה ביצוע **Shift** (במקום Reduce) כאשר המנתח נתקל ב-else, ובכך משייך אותו ל-if הפנימי.

5. טיפול בשגיאות

התוכנית מטפלת בשגיאות ומחזירה קוד יציאה בהתאם לדרישות:

- **שגיאות לקסיקליות:** מזוהות ב-Flex (על ידי הכלל . שתופס כל תו לא מוכר). התוכנית מדפיסה הודעת Lexical error עם התו והשורה, ומסיימת עם קוד יציאה 1.
- **שגיאות תחביריות:** מזוהות ב-Bison. הפונקציה yyerror מדפיסה הודעת Syntax error עם הלקסמה הנוכחית ומספר השורה, והתוכנית מסיימת עם קוד יציאה 2.